TripWire: A Deterministic Runtime Enforcement Layer for Secure AI-Agent Execution

Cryptographic Message Integrity, Capability-Based Authorization, Provenance Validation, and Default-Deny Containment

1. Abstract

Autonomous AI agents are increasingly entrusted with executing tools, accessing sensitive data, and interacting with external systems. Traditional security models rely on prompt-level instructions or the model's own safety judgments, which are demonstrably vulnerable to prompt injection, confused-deputy attacks, and adversarial manipulation. This paper introduces **TripWire**, a deterministic runtime security gateway that physically separates AI execution from system authorization. TripWire enforces a default-deny architecture through cryptographic message authentication, replay resistance, capability-based signed task manifests, and strict data-scope validation. By independently verifying the provenance and intent of every agent action against a cryptographic policy, TripWire successfully neutralizes unauthorized tool invocations and data exfiltration attempts, even when the underlying agent is fully compromised.

2. Introduction

As AI systems transition from conversational interfaces to autonomous agents, their security footprint expands dramatically. Agents now have the capability to invoke APIs, read proprietary databases, and send emails on behalf of users. When an agent processes untrusted external data—such as a malicious invoice or a targeted email—attackers can inject instructions that manipulate the agent into misusing its privileges (the "confused deputy" problem).

Current defenses primarily rely on model alignment and prompt engineering. However, expecting an LLM to reliably distinguish between a benign instruction and a malicious injection is fundamentally flawed. The model's reasoning cannot serve as an independent authorization boundary.

TripWire introduces a secondary enforcement layer. It operates on the principle that **the LLM proposes; the enforcement layer decides**. By interposing a deterministic, cryptographic gateway between the agent and its tools, TripWire ensures that no action is executed unless it strictly adheres to a verifiable policy, regardless of what the agent believes it should do.

Contributions

1. **Cryptographic Protocol Enforcement**: Implementation of HMAC-SHA256 message authentication, nonce tracking, and sequence enforcement for AI agent messaging.
2. **Capability-Based Manifests**: A signed task manifest system that explicitly scopes allowed tools, data, and destinations.
3. **Provenance-Aware Authorization**: Separation of trust levels between system policy, user tasks, and untrusted document content.
4. **Dynamic Firewall Containment**: A state-machine-driven risk engine that automatically triggers system lockdown upon detecting intrusion pressure.

3. Problem Statement and Motivation

AI agents are vulnerable to several classes of attacks when operating without a deterministic runtime gateway:

- **Prompt Injection from Documents**: Attacker objective: Execute unauthorized commands. Baseline: Agent blindly follows injected instructions. TripWire: Blocks via `UNTRUSTED_AUTHORITY` check.
- **Message Replay**: Attacker objective: Re-execute a previously authorized action (e.g., a payment). Baseline: Action repeats. TripWire: Blocks via `REPLAY` check using nonces and a 30-second freshness window.
- **Data Exfiltration**: Attacker objective: Send internal data to an external server. Baseline: Agent leaks data via an API tool. TripWire: Blocks via `DESTINATION_NOT_ALLOWED` and `DATA_OUT_OF_SCOPE`.
- **Forged Messaging**: Attacker objective: Spoof a message from a trusted agent to a vulnerable component. Baseline: Recipient processes spoofed message. TripWire: Blocks via `BAD_MAC` signature validation.

4. Threat Model

Assets

- Sensitive business data (e.g., customer PII, internal records).
- Tool execution capabilities (e.g., payments, emails).
- Audit history integrity.

Adversaries

- **External Attacker**: Injects malicious instructions via documents (e.g., invoices).
- **Compromised Agent**: Attempts to spoof messages, bypass restrictions, or access unauthorized tools.
- **Network Adversary**: Attempts to replay or manipulate inter-agent communication.

Trust Boundaries (TCB)

The Trusted Computing Base includes the Message Gateway, the Firewall, the Manifest Authority, and the Audit Logger. The AI Agents themselves are strictly **outside** the TCB.

Assumptions

- Cryptographic keys (HMAC secrets) are securely distributed and not leaked to the agents.
- The host Python runtime is not compromised.
- All sensitive tool invocations are routed strictly through the TripWire gateway (no bypass).

5. System Architecture

TripWire implements a centralized gateway architecture. Untrusted external input is processed by Agent 1 (Monitor), which proposes actions to the Gateway. The Gateway applies protocol and capability checks before routing to Agent 2 (Validator) or executing the Protected Tool.

(See `system_architecture.mmd`)

6. Protocol and Envelope Specification

TripWire enforces a strict message protocol (`tripwire.protocol`).

- **Signature**: `tag = HMAC-SHA256(key, canonical_message)`
- **Fields**: Sender, recipient, type, nonce, timestamp, sequence, payload.
- **Integrity**: Any tampering with the payload or headers results in a different canonical string, failing the HMAC check.

7. The Two-Layer Enforcement Model

Layer A — Protocol and Envelope Enforcement

- **A1. Schema validation**: Enforces structure.
- **A2. Cryptographic authentication (`BAD_MAC`)**: Validates HMAC signature.
- **A3. Freshness & Replay (`STALE` / `REPLAY`)**: Requires timestamps within 30s and tracks nonces.
- **A4. Sequence enforcement (`SEQUENCE_REGRESSION`)**: Drops out-of-order sequence IDs.
- **A5. Route enforcement (`UNAUTHORIZED_ROUTE`)**: Validates sender-recipient pairs.
- **A6. Rate limiting (`RATE_LIMITED`)**: Token bucket (burst 12, rate 2/sec).

Layer B — Task Capability Enforcement

- **B1. Signed Task Manifest**: Cryptographic policy defining allowed actions.
- **B2. Provenance (`UNTRUSTED_AUTHORITY`)**: Blocks instructions originating from untrusted document zones.
- **B3. Data scope (`DATA_OUT_OF_SCOPE`)**: Restricts file/record access to authorized spans.
- **B4. Destination (`DESTINATION_NOT_ALLOWED`)**: Enforces endpoint allowlists.
- **B5. Intent matching (`INTENT_MISMATCH`)**: Evaluates semantic alignment.
- **B6. Canary tokens (`CANARY_EGRESS`)**: Detects honeytokens in outgoing payloads.

8. Default-Deny Decision Model

The gateway evaluates decisions deterministically: `ALLOW(m, p) iff SchemaValid(m) ∧ Authenticated(m) ∧ Fresh(m) ∧ NotReplayed(m) ∧ SequenceValid(m) ∧ RouteAllowed(m) ∧ RateAllowed(m) ∧ ManifestValid(p) ∧ ProvenanceAllowed(m, p) ∧ DataScopeAllowed(m, p) ∧ DestinationAllowed(m, p) ∧ IntentAllowed(m, p)`

If any predicate fails, the result is `BLOCK` . Certain high-risk but valid actions result in `REQUIRE_APPROVAL` .

9. Firewall State Machine and Risk Scoring

States: `NORMAL` → `BLOCKING` → `LOCKDOWN` . Risk is calculated incrementally. When intrusion pressure crosses 100/200, the system escalates to `BLOCKING` , denying all tool executions. `LOCKDOWN` requires a manual, cryptographic operator recovery code to clear.

10. Audit Integrity and Forensic Evidence

The system maintains an HMAC-chained audit log. `H_i = HMAC(K_audit, H_(i−1) || canonical(event_i))` This ensures forensic tamper evidence; any modification or deletion breaks the hash chain.

11. Attack Scenarios and Adversarial Evaluation

TripWire includes a suite of built-in adversarial tests (`tripwire.attacks`).

1. **Forged Sender**: Agent 3 attempts to spoof Agent 1. Result: `BAD_MAC` .
2. **Replay Attack**: Agent 3 replays a valid sequence. Result: `REPLAY` .
3. **Flood Attack**: Agent 3 spams requests. Result: `RATE_LIMITED` .
4. **Document Injection**: Malicious invoice instructs data export. Result: `UNTRUSTED_AUTHORITY` .

12. Before-and-After Security Analysis

Baseline: Agent-Direct Execution

Untrusted Content → AI Agent → Tool Execution (Unrestricted). A malicious document easily manipulates the agent into executing arbitrary tools.

Protected: TripWire-Mediated Execution

Untrusted Content → AI Agent → TripWire Gateway (Deny Default) → Tool. Even if the agent is manipulated, the Gateway's provenance checks (`UNTRUSTED_AUTHORITY`) and data scope limits (`DATA_OUT_OF_SCOPE`) block the execution.

13. Quantitative Evaluation and Metrics

- **Tests Executed**: 214 integration and unit tests.
- **Attack Rejection Rate**: 100% (All simulated network and capability attacks correctly blocked). *(See `EVALUATION_REPORT.md`)*

14. Security Analysis

TripWire provides strong assurances against capability escalation and message forgery. It relies heavily on the assumption that the Python process and HMAC keys remain secure. It mitigates prompt injection not by "fixing" the LLM, but by stripping the LLM of the authority to unilaterally execute sensitive commands based on untrusted inputs.

15. Limitations and Open Research Questions

- **Gateway Bypass**: If a tool is accidentally exposed outside the gateway, TripWire offers no protection.
- **Semantic Ambiguity**: The gateway cannot detect subtle logical errors in instructions that technically adhere to the policy manifest.
- **Denial of Service**: An attacker could intentionally trigger a `LOCKDOWN` to disrupt legitimate business operations.

16. Deployment and Integration Guidance

Organizations should deploy TripWire as a sidecar or proxy between their agent execution environment and internal APIs. Strict secret management must be enforced for HMAC keys.

17. Conclusion

TripWire demonstrates that secure AI-agent execution requires moving authorization out of the language model and into a deterministic, policy-enforcing runtime layer. Through cryptographic message integrity and capability-based authorization, TripWire provides a robust defense against confused-deputy attacks and prompt injection.

18. References

1. OWASP Top 10 for LLM Applications (2025).
2. MITRE ATLAS (Adversarial Threat Landscape for AI Systems).
3. Capability-based Computer Systems (Levy, 1984).